



**UNIVERSIDAD
NACIONAL DE
INGENIERÍA**



TRANSFORMACIÓN
digital

CURSOS GRATUITOS

OTI UNI

Ansible desde Cero: Automatiza Linux y Servidores

Cursos PIT – 9na Edición

Instructor:  **Anthony Gonzalo Quispe Cordova**

Oficina de Tecnologías de la Información (OTI)

30 de mayo de 2026



Flujo de Cursos Orientado a DevOps



Horarios de los Cursos del Flujo DevOps



Fundamentos de Linux



Sábados



9:00 a.m. – 12:00 p.m.



23/05 – 27/06 | 18 h



Git y GitHub



Lun, Mar, Jue, Vie



4:00 – 7:00 p.m.



19/05 – 28/05 | 18 h



Fundamentos de Docker



Lun, Mar, Vie



4:00 – 7:00 p.m.



29/05 – 09/06 | 18 h



Ansible: Automatización



Sábados



2:00 – 5:00 p.m.



23/05 – 27/06 | 18 h

Índice de Contenidos

- 1 Bloque 1: ¿Qué es Ansible?
- 2 Bloque 2: Introducción a YAML
- 3 Bloque 3: Preparación del Laboratorio
- 4 Bloque 4: Comandos Ad-Hoc
- 5 Bloque 5: Primer Playbook

Bloque 1: ¿Qué es Ansible?

El Problema: Administración Manual de Servidores

- Instalar paquetes **uno por uno** en cada servidor.
- Configurar servicios **manualmente** y rezar para no olvidar un paso.
- Repetir **exactamente** lo mismo en 10, 50 o 100 máquinas.
- Un error humano → horas de depuración.

Resultado

Procesos lentos, inconsistentes y propensos a errores.



¿Te suena familiar?

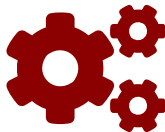
La Solución: Ansible

¿Qué es Ansible?

Motor de **automatización de TI** de código abierto, creado por Michael DeHaan (2012) y mantenido por **Red Hat**.

¿Para qué sirve?

- **Gestión de configuración** de servidores.
- **Despliegue** de aplicaciones de forma repetible.
- **Orquestación** de tareas en múltiples máquinas.



"Documentación ejecutable."

Ansible en Contexto

Origen

Ansible fue creado en **2012** por Michael DeHaan con una idea central: automatizar servidores de forma simple, sin instalar agentes remotos.

- En **2015**, Red Hat adquiere Ansible.
- Evoluciona de Ansible Tower a **Ansible Automation Platform**.
- Hoy automatiza Linux, Windows, redes, cloud y seguridad.



Simplicidad

+ SSH + YAML + módulos

Scripts vs Ansible

Scripts tradicionales	Ansible
Ejecutan pasos de forma secuencial.	Declara el estado deseado .
Requieren controlar muchos casos manualmente.	Usa módulos preparados para tareas comunes.
Pueden ser difíciles de reutilizar en equipos grandes.	Playbooks legibles y versionables.
El resultado depende mucho de cómo se escribió el script.	Busca ser idempotente : repetir no debería romper.

Ansible no reemplaza todos los scripts; los organiza dentro de una automatización más clara, auditable y repetible.

¿Por qué elegir Ansible?

Agentless

No requiere instalar nada
en los servidores remotos

YAML legible

Playbooks en lenguaje
simple y entendible

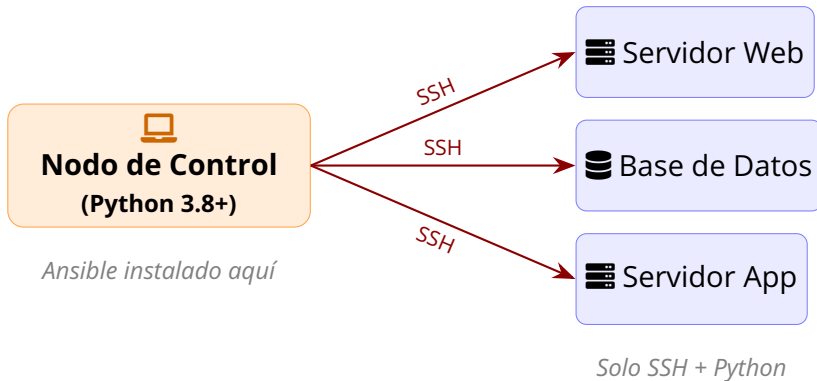
Solo necesita SSH

Conexión segura estándar
ya disponible en Linux

Idempotente

Solo aplica cambios si el
sistema no está en
el estado deseado

Arquitectura Agentless de Ansible



Modelo Push: Ansible empuja configuraciones vía SSH. Sin agentes ni demonios en los servidores = menor superficie de ataque.

Bloque 2: Introducción a YAML

Formatos de Datos: XML, JSON y YAML

XML	JSON	YAML
Muy formal, basado en etiquetas.	Ligero, común en APIs y aplicaciones web.	Legible para humanos, usado en automatización.
Verboso para archivos de configuración.	Usa llaves, corchetes y comillas con frecuencia.	Usa indentación, listas y pares clave-valor.

¿Por qué YAML en Ansible?

Los playbooks deben ser fáciles de leer, revisar y versionar. YAML permite expresar tareas, variables y estructuras sin demasiado ruido visual.

Reglas Básicas de YAML

Ideas clave

- Los comentarios empiezan con #.
- La estructura depende de la **indentación**.
- Se recomiendan **2 espacios**; no usar tabs.
- Las listas usan guion: - item.
- Las variables suelen ser clave: valor.

Ejemplo corto

```
# Datos del servidor
servidor: web01
puerto: 80
activo: true
paquetes:
  - nginx
  - curl
```

Secuencias, Mapas y Escalares

Escalares

Valores simples: texto, números, booleanos o nulos.

Mapas

Pares clave: valor, parecidos a un diccionario.

Secuencias

Listas ordenadas de elementos.

Mezcla común

```
usuario:
  nombre: ansible
  sudo: true
  grupos:
    - admins
    - devops
contacto: {ciudad: Lima, pais: Peru}
```

Estructura Común de un Playbook

Playbook mínimo

```
---  
- name: Preparar servidor web  
  hosts: web  
  become: true  
  tasks:  
    - name: Instalar nginx  
      apt:  
        name: nginx  
        state: present
```

- `hosts`: indica el grupo del inventario.
- `become`: permite ejecutar con privilegios

Mini Actividad: Autobiografía en YAML

Crea un archivo `autobiografia.yaml` para practicar estructura e indentación.

Plantilla sugerida

```
---
# Autobiografia en YAML
nombre: "Tu nombre"
contacto: {correo: "tu@correo.com", ciudad: "Lima"}
habilidades:
  - Linux
  - Git
  - Ansible
metas:
  corto_plazo: "Practicar automatizacion"
  largo_plazo: "Administrar servidores"
# Fin del documento
```

Bloque 3: Preparación del Laboratorio

Herramientas Necesarias para el Laboratorio

VirtualBox y Ubuntu

Usaremos una **única máquina virtual** como entorno aislado principal para nuestro laboratorio.

Docker + Dive Into Ansible

Dentro de la VM, Docker levantará los múltiples nodos (*contenedores*) que gestionaremos con Ansible.



VM Ubuntu (VirtualBox)



Contenedores (Nodos)

Instalación de Ansible por Sistema Operativo

Ubuntu / Debian

```
sudo apt update  
sudo apt install ansible -y
```

RHEL / CentOS / Fedora

```
sudo dnf install ansible -y
```

macOS

```
brew install ansible
```

En nuestro laboratorio

El contenedor ubuntu-c ya trae Ansible instalado; solo verificaremos con `ansible -version`.

Arquitectura del Laboratorio Multi-Servidor



Objetivo

Simular un entorno real con múltiples servidores para practicar Ansible sin riesgo.

Preparación del Entorno Base

1. Instalar VirtualBox

- Descargar e instalar VirtualBox en tu computadora anfitriona.

2. Crear la VM Ubuntu Base

- Descargar la ISO de **Ubuntu Server 22.04** o superior.
- Crear una VM con al menos **2 GB de RAM** y **2 CPUs**.
- Configurar el adaptador de red en modo **Adaptador Sólo Anfitrión o Red Interna** para tener conexión con la VM.
- Instalar el sistema operativo.

Levantando el Laboratorio en la VM

Pasos dentro de la VM Ubuntu:

- 1 Instalar Docker y Git: `sudo apt update && sudo apt install docker.io docker-compose-v2 git -y`
- 2 Clonar el repositorio: `git clone https://github.com/spurin/diveintoansible-lab.git`
- 3 Iniciar el laboratorio: `cd diveintoansible-lab && sudo docker compose up -d`

Importante

Esto creará contenedores interconectados. El nodo `ubuntu-c` actuará como tu controlador de Ansible.

Verificando el Entorno

El laboratorio de *diveintoansible* ya viene con **Ansible instalado** y las **llaves SSH preconfiguradas** entre el nodo de control y los demás nodos.

Ingresar al nodo de control y verificar:

Comandos

```
sudo docker exec -it ubuntu-c bash
ansible --version
ssh ubuntu1
```

¡Estamos listos para empezar a usar Ansible inmediatamente sin configuraciones manuales!

Bloque 4: Comandos Ad-Hoc

¿Qué son los Comandos Ad-Hoc?

Definición

Comandos **únicos** que se ejecutan directamente desde la terminal sin necesidad de escribir un playbook.

¿Cuándo usarlos?

- Tareas rápidas y puntuales.
- Verificar estado de servidores.
- Ejecutar un comando en múltiples máquinas al instante.



*“Una línea,
múltiples servidores.”*

El Archivo de Inventario

El inventario define **qué servidores** gestiona Ansible.

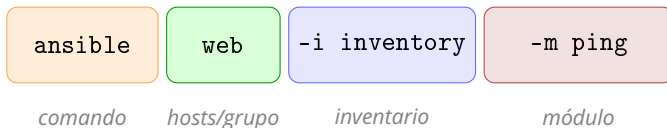
Inventario (inventory.ini)

```
[web]
ubuntu1
ubuntu2

[db]
centos1
centos2

[all:vars]
ansible_user=ansible
```

Sintaxis de Comandos Ad-Hoc



Estructura general

```
ansible <hosts> -i <inventario> -m <módulo> [-a «argumentos>"]
```

Ejemplos Prácticos de Comandos Ad-Hoc (1/3)

1. Verificar conectividad (módulo ping)

```
ansible all -i inventory.ini -m ping
```

Nota: No es un ping ICMP. Verifica que Ansible pueda abrir una sesión SSH exitosamente.

2. Ejecutar un comando remoto (módulo command)

```
ansible web -i inventory.ini -m command -a "uptime"
```

Ejemplos Prácticos de Comandos Ad-Hoc (2/3)

3. Instalar un paquete (módulo apt)

```
ansible web -i inventory.ini -m apt \
  -a "name=htop state=present" --become
```

4. Copiar un archivo a los servidores

```
ansible all -i inventory.ini -m copy \
  -a "src=./hola.txt dest=/tmp/hola.txt"
```

Ejemplos Prácticos de Comandos Ad-Hoc (3/3)

5. Recopilar información del sistema (facts)

```
ansible web -i inventory.ini -m setup
```

6. Reiniciar un servicio

```
ansible web -i inventory.ini -m service \
  -a "name=nginx state=restarted" --become
```

Módulos Más Usados en Ansible

Módulo	Descripción	Ejemplo
ping	Verificar conectividad	-m ping
command	Ejecutar comando	-a 'whoami'
shell	Ejecutar con shell	-a 'cat /etc/os-release'
copy	Copiar archivos	-a 'src=... dest=...'
apt	Gestión de paquetes	-a 'name=nginx state=present'
service	Gestión de servicios	-a 'name=nginx state=started'
setup	Recopilar facts	-m setup

Ansible tiene **miles de módulos** para casi cualquier tarea de administración.

Bloque 5: Primer Playbook

De Comandos Ad-Hoc a Playbooks

Comandos ad-hoc

- Ideales para pruebas y tareas rápidas.
- Se ejecutan una vez desde la terminal.

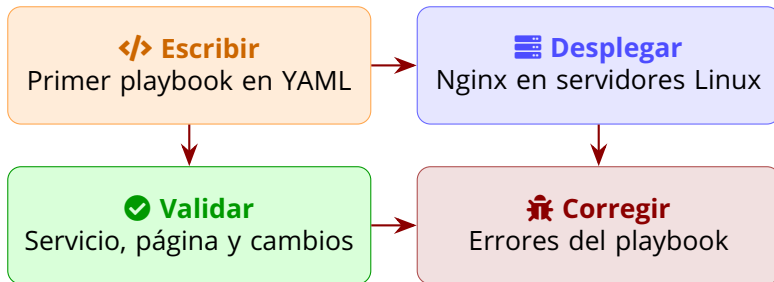
Playbooks

- Guardan la automatización en archivos YAML.
- Permiten repetir, revisar y versionar procedimientos.



**Automatización como
código**

Objetivo de la Sesión 2



¿Qué es un Playbook?

Definición

Un **playbook** es un archivo YAML que indica qué hosts serán gestionados y qué tareas ejecutará Ansible para llevarlos a un estado deseado.

Estructura mínima

```
---
- name: Mi primer playbook
  hosts: web
  become: true
  tasks:
    - name: Verificar conectividad
      ansible.builtin.ping:
```

Partes Principales de un Playbook

 **name**

Nombre legible del play

 **hosts**

Grupo o servidor objetivo

 **become**

Ejecutar con privilegios

 **tasks**

Lista de tareas

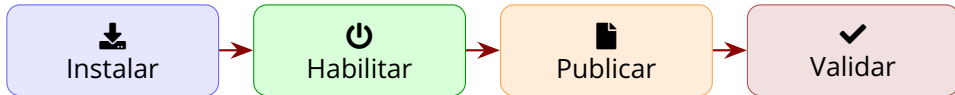
Anatomía de una Tarea

Una tarea = nombre + módulo + parámetros

```
- name: Instalar Nginx
  ansible.builtin.apt:
    name: nginx
    state: present
    update_cache: true
```

- `name`: describe qué hace la tarea.
- `ansible.builtin.apt`: módulo usado para gestionar paquetes.
- `state: present`: declara el estado deseado.

Flujo de Despliegue de Nginx



Meta

Dejar un servidor web funcionando con una página personalizada usando un solo comando de Ansible.

Playbook: Instalar y Habilitar Nginx

nginx.yml

```
---
- name: Desplegar Nginx
  hosts: web
  become: true
  tasks:
    - name: Instalar Nginx
      ansible.builtin.apt:
        name: nginx
        state: present
        update_cache: true

    - name: Asegurar que Nginx esté activo
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: true
```


Publicar una Página HTML

Agregar una tarea con copy

```
- name: Publicar página principal
  ansible.builtin.copy:
    dest: /var/www/html/index.html
    content: |
      <h1>Servidor gestionado con Ansible</h1>
      <p>Despliegue automatizado de Nginx.</p>
    owner: www-data
    group: www-data
    mode: '0644'
```

Idempotencia: la Idea Clave

¿Qué significa?

Ejecutar el mismo playbook varias veces debe producir el **mismo estado final**, sin repetir cambios innecesarios.

- Si Nginx ya está instalado, Ansible no lo reinstala.
- Si el servicio ya está activo, no lo reinicia sin motivo.
- Si el archivo no cambió, no vuelve a copiarlo.



Repetible y seguro

Cómo Leer el Resultado de Ansible

Estado	Significado	Lectura rápida
ok	Ya estaba correcto	No hubo cambio
changed	Ansible modificó algo	Cambio aplicado
failed	La tarea falló	Revisar error
skipped	No se ejecutó	Condición no cumplida
unreachable	No conectó al host	Revisar SSH/inventario

La segunda ejecución de un buen playbook debería mostrar menos changed.

Ejecutar el Playbook

Comandos útiles

```
ansible-playbook -i inventory.ini nginx.yml
```

```
ansible-playbook -i inventory.ini nginx.yml --check
```

```
ansible-playbook -i inventory.ini nginx.yml -v
```

- Primera línea: aplica el playbook.
- `-check`: simula los cambios antes de ejecutarlos.
- `-v`: muestra más detalle para diagnosticar errores.

Validar el Despliegue

Desde el nodo de control

Verificamos que el servicio responda y que la página publicada esté disponible.

Validación rápida

```
ansible web -i inventory.ini -m command -a "systemctl status nginx"
ansible web -i inventory.ini -m uri -a "url=http://localhost"
```

- `systemctl status nginx`: confirma el estado del servicio.
- `uri`: prueba una respuesta HTTP desde el servidor.

Errores Básicos y Cómo Corregirlos

Error común	Causa probable	Qué revisar
UNREACHABLE	SSH o inventario	IP, usuario, llave
Error de YAML	Indentación incorrecta	Espacios y dos puntos
Permiso denegado	Falta privilegio	become: true
Paquete no encontrado	Caché desactualizada	update_cache
Servicio no inicia	Configuración inválida	Logs de Nginx

Mini Laboratorio de la Sesión 2

- 1 Crear el archivo `nginx.yml`.
- 2 Ejecutarlo contra el grupo `web`.
- 3 Ejecutarlo una segunda vez y comparar `ok` vs `changed`.
- 4 Cambiar el contenido del HTML y volver a ejecutar.
- 5 Validar el sitio con `curl` o el módulo `uri`.

Comando final

```
ansible-playbook -i inventory.ini nginx.yml
```

Resumen de la Sesión 2

- 1 Un **playbook** describe tareas en YAML para automatizar servidores de forma repetible.
- 2 Usamos `hosts`, `become` y `tasks` para estructurar la automatización.
- 3 Desplegamos **Nginx** instalando el paquete, habilitando el servicio y publicando HTML.
- 4 Entendimos la **idempotencia**: ejecutar varias veces sin repetir cambios innecesarios.
- 5 Practicamos ejecución, validación y corrección de errores básicos en playbooks.

Próxima Sesión

Sesión 3: Inventarios y Automatización Multi-Servidor — `hosts`, grupos, variables y MariaDB.